

FinGPT Part 2 Development Guide

This document describes the repository as it exists now, with an emphasis on the real execution workflow, each LLM call, and where data is transformed or filtered.

Purpose

This repo implements a local, inference-only financial news pipeline with two agents:

1. **Agent 1** converts raw news text into a structured **NewsFingerprint**.
2. **Agent 2** converts the fingerprint into a trading direction.
3. The **backtest** package runs the same logic over a labeled dataset and compares predictions against realized returns.

The central design choice is that model decisions are taken from real token log-probabilities read from vLLM **prompt_logprobs**, not from self-reported scores written by the model.

End-To-End Workflow

There are two main execution modes.

Live / fetch mode

Entry point: [pipeline.py](#)

Flow:

1. **run_pipeline(tickers, limit, as_of_timestamp)** is called.
2. **fetch_recent_articles(...)** pulls candidate articles from Alpaca or Finnhub.
3. The fetch layer applies leakage-safe filtering:
 - keep only **created_at** $\leq$ **as_of_timestamp**
 - drop articles with missing / unparseable timestamps
 - keep only the latest article per requested ticker
4. For each retained article:
 - **article_text** = **headline** + **summary**
 - **extract_fingerprint(article_text, ticker=..., headline=...)**
 - **generate_signal(fingerprint)**
5. Valid signals are written to **output/signals_<timestamp>.json**.

Important live-data rule:

- **ticker** is external metadata from the fetch layer.
- **headline** is external metadata from the fetch layer.
- **Agent 2** does not re-read the article body.

Backtest mode

Entry point: [backtest/backtester.py](#)

Flow:

1. Load a dataset with `load_dataset(...)` or a local CSV/parquet file.
2. Normalize columns to `input`, `output`, `answer`, `ticker`.
3. Build rows with:
 - `ticker`
 - `headline`
 - `article_text`
 - `start_date`
 - `end_date`
 - `fingpt_label`
4. Process rows in batches:
 - Agent 1 batch extraction
 - Agent 2 batch signal generation
 - realized return fetch
 - row flattening to CSV
5. Write detailed results to `output/backtest_results.csv` or the CLI-selected path.

Important backtest rule:

- For Hugging Face multi-news prompts, the parser takes the first headline as the external `headline`.
- Dataset `ticker` is passed directly to Agent 1 as the authoritative ticker.

Data Contracts

NewsFingerprint

Defined in `agent1/schema.py`.

Fields:

- External / metadata fields:
 - `ticker`
 - `source`
 - `published_at`
 - `headline`
 - `companies_named`
 - `event_keywords`
- Sentiment fields:
 - `sentiment_label`
 - `sentiment_score`
 - `sentiment_confidence`
 - `sentiment_probabilities`
 - `sentiment_logits`
 - `calibration_T`
- Event-type fields:
 - `event_type`
 - `event_type_confidence`
 - `event_type_margin`

- `event_type_method`
- `event_type_logits`
- `event_type_probabilities`
- `secondary_event_type`
- `secondary_event_type_confidence`
- Pass-through field:
 - `article_text`

Important behavior:

- `ticker` is no longer inferred from `companies_named[0]`.
- `headline` is now preserved from upstream pipeline inputs when provided.
- `source` and `published_at` are normalized to strings even if the extraction model returns a list.
- `companies_named` may be empty, but `_assemble_fingerprint(...)` will fill it from `ticker` when possible.
- `event_keywords` is retained only for backward compatibility; it is set to `[event_type.lower()]` during fingerprint assembly.

TradingSignal

Defined in [agent2/schema.py](#).

Fields:

- `ticker`
- `direction`
- `strategy_tag`
- `confidence`
- `cot`
- `signal_logits`
- `raw_signal_logits`
- `pmi_null_logprobs`
- `pmi_alpha_used`
- `signal_probabilities`
- `calibration_T`
- `signal_filter_forced_hold`
- `signal_filter_reason`

Important behavior:

- `strategy_tag` is always `"event_driven"`.
- `cot` is empty in no-CoT mode.

Every LLM Call

This section is the most important one for development work.

Agent 1 call 1: guided fact extraction

Code path:

- `agent1/extractor.py`: `_build_extraction_prompt`, `_generate_extraction_text`, `_parse_extraction_or_fallback`

Prompt source:

- `agent1/prompt.py`: `EXTRACTION_PROMPT`

Input:

- raw `article_text`

Requested structured fields:

- `source`
- `published_at`
- `companies_named`
- `event_keywords`

Mechanism:

1. Build a prompt from `EXTRACTION_PROMPT` + `article_text`.
2. Ask vLLM to generate JSON under a guided schema.
3. Try to parse:
 - first as JSON
 - then as balanced JSON blob
 - then as markdown-style extracted sections

Fallback behavior:

- If parsing still fails, use an empty extraction payload instead of dropping the row immediately.
- This allows the row to continue through sentiment and event-type scoring.

Why this is safe:

- `source` is not consumed by Agent 2.
- `headline` is provided externally when available.
- `ticker` is provided externally when available.
- `companies_named` can be repaired from `ticker`.

Agent 1 call 2: sentiment scoring

Code path:

- `agent1/extractor.py`: `_score_sentiment`, `_process_sentiment_result`

Prompt source:

- `agent1/prompt.py`: `SENTIMENT_PROMPT`

Input:

- raw `article_text`

Output token set:

- POSITIVE
- NEGATIVE
- NEUTRAL

Mechanism:

1. Build a direct sentiment classification prompt.
2. Call `get_real_choice_logits(...)` or `get_real_choice_logits_batch(...)` with:
 - `decision_prefix = "Sentiment: "`
 - `use_cot = False`
3. Read the real next-token log-probability for each class label.
4. Convert logits to probabilities with `softmax(logits / CALIBRATION_T)`.
5. Set:
 - `sentiment_label`
 - `sentiment_confidence`
 - `sentiment_probabilities`
 - `sentiment_logits`

Fallback behavior:

- If logits parsing fails, use:
 - uniform probabilities
 - NEUTRAL
 - zero logits

Agent 1 call 3: event-type scoring

Code path:

- `agent1/extractor.py: _score_event_type, _process_event_type_result`

Prompt source:

- `agent1/prompt.py: EVENT_TYPE_PROMPT`

Input:

- raw `article_text`

Output token set:

- A, B, C, D, E, F, G

Mapping:

- A -> EARNINGS
- B -> GUIDANCE
- C -> ANALYST_RATING
- D -> LEGAL_REGULATORY
- E -> MNA

- F -> PRODUCT_BUSINESS
- G -> MACRO

Mechanism:

1. Build the event-type prompt.
2. Call `get_real_choice_logits(...)` or `_batch(...)` with:
 - `decision_prefix = "Answer: "`
 - `use_cot = False`
3. Read real next-token log-probabilities for A-G.
4. Compute calibrated probabilities.
5. Rank top-1 and top-2.
6. Apply rule-based filtering:
 - if `top_prob < FINGPT_EVENT_TYPE_MIN_CONFIDENCE`, assign OTHER
 - if `margin < FINGPT_EVENT_TYPE_MIN_MARGIN`, assign OTHER
 - else accept the mapped label

Important design choice:

- OTHER is never model-scored.
- OTHER is assigned only by Python post-processing.

Agent 2 call 1: optional null-context PMI prior

Code path:

- `agent2/reasoner.py: _compute_null_logprobs`

Purpose:

- Compute the language-model prior over strategy score tokens without real news context.

Mechanism:

1. Build a synthetic neutral `NewsFingerprint`.
2. Build the Agent 2 prompt.
3. Score A/B/C logits.
4. Save them in memory and optionally on disk.

This call happens once per model/config combination, not once per article.

Agent 2 call 2: strategy scoring

Code path:

- `agent2/reasoner.py: _build_prompt, generate_signal, generate_signal_batch, _process_signal_result`

Prompt source:

- `agent2/prompt.py`

Input:

- `ticker`
- `headline`
- Agent 1 sentiment outputs
- Agent 1 event-type outputs
- `companies_named`

Notably excluded:

- full `article_text`
- raw article summary/body
- event keyword extraction

Output token set:

- `A, B, C`

Mapping:

- `A -> BUY -> long`
- `B -> HOLD -> neutral`
- `C -> SELL -> short`

Mechanism:

1. Build the compact fingerprint-only prompt.
2. If `FINGPT_SIGNAL_USE_COT=True`, first generate CoT and then score A/B/C.
3. Otherwise score A/B/C directly.
4. Optionally apply PMI prior correction:
 - `adjusted = raw - pmi_alpha * null`
5. Convert to calibrated probabilities with `softmax(adjusted / CALIBRATION_T)`.
6. Apply signal filters:
 - low confidence -> force HOLD
 - low margin -> force HOLD
 - weak BUY -> force HOLD
 - weak SELL -> force HOLD
7. Assemble `TradingSignal`.

Batch Behavior

Agent 1 batch workflow

`extract_fingerprint_batch(article_texts, tickers=None, headlines=None)` performs:

1. One batched guided extraction call.
2. One batched sentiment scoring call.
3. One batched event-type scoring call.
4. Per-item fingerprint assembly in Python.

Per-item assembly can still succeed even when fact extraction parsing fails, because extraction now has an empty-payload fallback.

Agent 2 batch workflow

`generate_signal_batch(fingerprints)` performs:

- In no-CoT mode:
 - one batched A/B/C scoring call
- In CoT mode:
 - one batched CoT generation call
 - one batched A/B/C scoring call

Backtest batch workflow

Per batch:

1. Run Agent 1 batch extraction.
2. Keep only valid fingerprints.
3. Run Agent 2 on valid fingerprints.
4. Fetch realized returns row by row.
5. Flatten everything into CSV columns.

Failure / Degradation Modes

Fact extraction failure

Current behavior:

- malformed extraction JSON does not automatically kill the row
- fallback uses empty extraction payload
- row can still proceed if sentiment and event-type scoring succeed

Sentiment failure

Current behavior:

- downgrade to **NEUTRAL** with uniform probabilities

Event-type failure

Current behavior:

- assign **OTHER**
- mark `event_type_method = "event_type_logits_failed"`

Agent 2 scoring failure

Current behavior:

- return **None**
- mark the row as **signal_failed** in backtest

Configuration

Main configuration file: [config.py](#)

Core model / runtime

- FINGPT_MODEL_PATH
- SHARE_SINGLE_LLM_BETWEEN_AGENTS
- CALIBRATION_T
- LOGITS_MAX_TOKENS

News fetching

- NEWS_PROVIDER
- ALPACA_API_KEY
- ALPACA_API_SECRET
- ALPACA_DEFAULT_LIMIT
- FINGPT_NEWS_FETCH_COUNT
- FINNHUB_API_KEY
- FINNHUB_TIMEOUT_SEC
- FINNHUB_MAX_CALLS_PER_SEC
- FINNHUB_MAX_RETRIES
- FINNHUB_RETRY_BASE_DELAY_SEC

Agent 1 event-type filter

- FINGPT_EVENT_TYPE_MIN_CONFIDENCE
- FINGPT_EVENT_TYPE_MIN_MARGIN

Agent 2 PMI / signal filters

- FINGPT_PMI_ALPHA
- FINGPT_SIGNAL_MIN_CONFIDENCE
- FINGPT_SIGNAL_MIN_MARGIN
- FINGPT_BUY_THRESHOLD
- FINGPT_SELL_THRESHOLD
- FINGPT_SIGNAL_USE_COT

Backtest

- FINGPT_BACKTEST_STRICT_MODE
- FINGPT_BACKTEST_BATCH_SIZE

Backtest Outputs

Main output:

- CSV with one row per dataset example

Important exposed columns:

- dataset / metadata:
 - `ticker`
 - `headline`
 - `fingerprint_ticker`
 - `signal_ticker`
 - `start_date`
 - `end_date`
 - `fingpt_label`
- Agent 1:
 - `sentiment_*`
 - `event_type_*`
 - `event_logprob_*`
 - `event_prob_*`
- Agent 2:
 - `direction`
 - `confidence`
 - `raw_signal_logprob_*`
 - `pmi_null_logprob_*`
 - `pmi_adjusted_logit_*`
 - `signal_prob_*`
 - `signal_filter_*`
- evaluation:
 - `realized_return`
 - `strategy_return`
 - `skipped_reason`

Recommended Files To Edit

If you want to change extraction behavior:

- [agent1/prompt.py](#)
- [agent1/extractor.py](#)
- [agent1/schema.py](#)

If you want to change strategy behavior:

- [agent2/prompt.py](#)
- [agent2/reasoner.py](#)
- [agent2/schema.py](#)

If you want to change backtesting:

- [backtest/dataset_parser.py](#)
- [backtest/backtester.py](#)
- [backtest/price_fetcher.py](#)

If you want to change live fetching / leakage-safe selection:

- [ingestion/news_fetcher.py](#)

- [pipeline.py](#)